

Étude de dimensionnement : Parallélisation d'utilisateurs sur GPU

Axel Bröns¹ and Stanislas Boiton-Ledenko¹

¹Infogene

13 juillet 2026

Résumé

L'hébergement et l'inférence des grands modèles de langage (LLM) sous contraintes matérielles de VRAM constituent un défi majeur pour le déploiement multi-utilisateurs. Cet article présente une étude comparative et une formalisation mathématique de l'occupation mémoire de deux architectures distinctes : **GPT-OSS-20B** (dense/sliding-window) et **Qwen3.6-35B-A3B** (hybride MoE/Gated DeltaNet), évaluées sur un processeur graphique NVIDIA RTX A6000 (48 Go). Nous modélisons rigoureusement la dynamique des différents postes de mémoire : le stockage statique des poids, l'infrastructure du moteur d'inférence, le pic de mémoire d'activation lors de la phase de prefill, et le budget dynamique alloué au KV cache. En analysant les paradigmes d'allocation d'Ollama (statique) et vLLM (dynamique via PagedAttention), nous démontrons théoriquement que l'architecture hybride de **Qwen3.6-35B-A3B** permet de maximiser la capacité globale et offre un contexte utilisateur 20% plus grand que **GPT-OSS-20B** à budget de KV cache équivalent. Ce travail fournit ainsi un cadre formel pour dimensionner les infrastructures de calcul et optimiser le taux de concurrence en production.

Table des matières

1	Introduction	3
2	Modèles étudiés	3
3	Mémoire d'infrastructure	4
4	Mémoire statique	4
4.1	gpt-oss-20b	5
4.2	Qwen3.6-35B-A3B	5
4.3	Comparaison entre les deux modèles	5
5	Mémoire d'activation	5
5.1	Formulation formelle de la mémoire d'activation	6
5.1.1	gpt-oss-20b	6
5.1.2	Qwen3.6-35B-A3B	6
5.2	Approximation pratique	7
6	Mémoire du KV cache	8
6.1	KV cache : formulation formelle	8
6.2	Approximation pratique	9
7	Capacité multi-utilisateurs	10
7.1	Modèle d'allocation statique	10
7.1.1	Détails formels pour la capacité multi-utilisateurs	10
7.1.2	Approximation en pratique	12
7.2	Impact réel du moteur d'inférence	12
8	Ollama et vLLM	12
8.1	PagedAttention	14
8.1.1	PagedAttention : formulation formelle	14
9	Conclusion et Perspectives	14
A	Notations	16

1 Introduction

Avec l'arrivée de l'optimisation GPU en ingénierie IA, nous voulons à tout prix optimiser la place des modèles pour avoir le moins de GPU possible mais aussi payer moins d'électricité lors de l'utilisation de celui-ci. Ainsi, pour répondre simplement à la question : “Qu'est-ce qu'un modèle génératif consomme lors de la mise en production sur un GPU?”, on peut séparer la VRAM mobilisée par modèles (avec attention [5] ou attention linéaire principalement) en quatre parties :

- **Mémoire statique** (poids du modèle)
- **Mémoire d'infrastructure** (Overhead du framework, généralement fixe par GPU)
- **Mémoire dynamique** (KV Cache)
- **Mémoire d'activation** (allouée temporairement suivant les calculs)

Ainsi, intuitivement on peut penser, plus le modèle est gros plus il va consommer. C'est juste mais faux en même temps, c'est souvent une approximation qui fonctionne lorsqu'on compare deux modèles de taille très différente. Mais si nous voulons être le plus rigoureux possible nous devons faire une étude de dimensionnement architecturale pour réellement évaluer la consommation d'un modèle mis en production avec X utilisateurs en même temps. C'est pour cette raison que nous avons comparé 2 modèles de poids différent mais avec des spécificités et architecture totalement différentes, ainsi la consommation est dans ce cas non trivial et peut être non intuitif.

2 Modèles étudiés

Dans la suite de ce papier, nous allons étudier les 2 modèles suivant :

1. [GPT-OSS-20B](#) [1]
2. [Qwen/Qwen3.6-35B-A3B](#) [2]

Les détails de l'architecture de GPT-OSS-20B proviennent directement du fichier `config.json` du dépôt Hugging Face et (surtout) de la model card publiée par OpenAI [1, 3]. Les paramètres de Qwen3.6-35B-A3B proviennent du `config.json` fourni et du blog officiel Qwen [2, 4].

Paramètre architectural	GPT-OSS-20B	Qwen3.6-35B-A3B
Paramètres totaux	20.91B	$\approx$ 33.79B estimés
Paramètres actifs	3.61B	$\approx$ 2.59B estimés
Architecture	Dense / hybride sliding-window	Hybride GDN + full attention + MoE
Couches totales	24	40
Couches full attention	12	10
Couches linéaires / sliding	12 sliding-window	30 Gated DeltaNet
Fenêtre locale	128 tokens	N/A
Têtes KV	8	2 (full att.)
Dimension d'une tête	64	256 (full att.)
Quantification	MXFP4	bfloat16
Taille du checkpoint	12.8 GiB ($\approx$ 13.76 Go)	non publiée
Contexte maximal	131 072 tokens	262 144 tokens
Multimodalité	Non indiquée	Oui (image et vidéo)

TABLE 1 – Comparaison architecturale entre `gpt-oss-20b` et `Qwen3.6-35B-A3B`

3 Mémoire d'infrastructure

Avant même de charger le moindre poids, le moteur d'inférence alloue en VRAM un espace de gestion incompressible. Cet overhead regroupe plusieurs composantes : les bibliothèques CUDA et les pilotes NVIDIA, les noyaux Triton compilés à la volée lors du premier appel, les structures internes du framework PyTorch (allocateur, graphes CUDA), ainsi que les métadonnées du moteur lui-même (tables de routage pour vLLM, contexte de session pour Ollama).

En pratique, cet overhead est de l'ordre de **1 à 1.5 Go** par instance de moteur, indépendamment de la taille du modèle servi. Il est constant durant toute la durée de vie du processus : ni le nombre de requêtes, ni la longueur des séquences ne l'affectent.

À noter que si plusieurs instances de moteur coexistent sur le même GPU (par exemple deux processus vLLM distincts, ou Ollama avec plusieurs modèles chargés simultanément), cet overhead s'additionne, chaque instance conserve son propre espace de gestion.

4 Mémoire statique

Cette partie concerne uniquement le chargement du modèle sur le GPU. En pratique, le GPU effectue ses calculs dans sa mémoire VRAM, qui sert de zone de travail pour les poids, les buffers et les structures temporaires du moteur d'inférence. La mémoire statique correspond à la part occupée par les poids du modèle : elle est fixe pour un modèle donné, à format de poids et moteur d'inférence identiques. On peut la réduire grâce à une **quantisation**¹. Il est important de noter que cette dernière doit être modérée car elle peut nuire aux performances du modèle.

1. La quantisation consiste à représenter les poids avec moins de bits. Par exemple, un poids stocké en précision flottante haute précision peut être compressé dans un format plus compact, ce qui réduit l'empreinte mémoire, mais peut dégrader légèrement la qualité.

4.1 gpt-oss-20b

D’après la documentation officielle d’OpenAI [1], GPT-OSS-20B occupe **12.8 GiB** (12.8 GiB $\approx$ 13.7 Go) en mémoire (disque ou VRAM). Le modèle est déjà quantifié sur les couches MLP (les plus importantes du modèle) en MXFP4, alors que les autres couches restent en (bfloat16).

4.2 Qwen3.6-35B-A3B

Pour Qwen3.6-35B-A3B, aucun format MXFP4 natif n’est publié dans la documentation de référence, on raisonne donc ici à partir des poids disponibles sur Hugging Face et des informations du modèle. Le modèle compte environ **33.79B** paramètres au total et occupe environ **70 Go** de mémoire en bfloat16. Un stockage naïf en bfloat16 dépasse donc clairement ce qu’une RTX A6000 peut accueillir en terme de mémoire, hors offload ou sharding. En pratique, le déploiement local doit donc reposer sur une version quantifiée, par exemple en Q4_K_M ou dans un format équivalent proposé par des distributions comme Unsloth. [2, 4]

4.3 Comparaison entre les deux modèles

Paramètre	GPT-OSS-20B	Qwen3.6-35B-A3B
Paramètres totaux	20.91B	$\approx$ 33.79B
Taille brute (bfloat16)	$\approx$ 13.7 Go	$\approx$ 70 Go
Quantification native	MXFP4 sur les poids MoE linéaires	Aucune quantification native documentée ici
Format de déploiement courant	Modèle natif publié par OpenAI	Version quantifiée recommandée pour un déploiement local
Taille après quantification	12.8 GiB	$\approx$ 20 Go selon le format quantifié
Tient sur une A6000 (48 Go)	Oui, sans difficulté majeure	Oui, uniquement en version quantifiée ou avec offload

TABLE 2 – Bilan de la mémoire statique pour gpt-oss-20b et Qwen3.6-35B-A3B

5 Mémoire d’activation

La mémoire d’activation correspond à la mémoire temporaire utilisée pendant les calculs. Elle sert à stocker les résultats intermédiaires des différentes opérations. Elle est ensuite libérée une fois le calcul terminé. Il reste néanmoins important de la prendre en compte, en particulier lors de l’inférence. On distingue donc deux modes d’utilisation : l’entraînement et l’inférence. Nous ne nous concentrons que sur cette dernière ici.

En pratique, la mémoire d’activation se décompose en deux phases :

- **Prefill** : phase pendant laquelle le modèle traite l’ensemble du prompt utilisateur. C’est généralement la phase la plus coûteuse en mémoire d’activation.
- **Decode** : phase pendant laquelle le modèle génère les tokens un par un. Son coût mémoire est généralement plus faible et souvent négligeable dans une première approximation.

5.1 Formulation formelle de la mémoire d'activation

Définition 5.1 (Tenseur d'activation). On appelle *tenseur d'activation* tout tenseur intermédiaire devant coexister en VRAM durant le forward pass d'une couche. Pour un batch de s tokens en précision BF16, tout tenseur de dimension d occupe

$$\tau(s, d) = s \times d \times p \quad \text{octets}, \quad (1)$$

avec $p = 2$ octets. La mémoire d'activation pic est atteinte sur la couche la plus coûteuse, que l'on note ℓ^* :

$$M_{\text{act}}(s) = \max_{\ell} \sum_k \tau(s, d_k^{(\ell)}), \quad (2)$$

où la somme porte sur l'ensemble des tenseurs k de la couche ℓ .

Remarque 5.2. On se place en inférence uniquement (pas de rétropropagation). Les tenseurs d'une couche sont libérés dès que cette couche a terminé son calcul. FlashAttention est supposé actif, ce qui supprime la matrice d'attention $s \times s$ de la mémoire.

5.1.1 gpt-oss-20b

Proposition 5.3 (Pic d'activation pour gpt-oss-20b). *Le pipeline d'une couche MoE de gpt-oss-20b est*

$$\text{Entrée} \xrightarrow{\text{RMSNorm}} \xrightarrow{\text{Attention}} \xrightarrow{\text{RMSNorm}} \xrightarrow{\text{MoE}} \text{Sortie}.$$

Le pic d'activation par token est

$$M_{\text{act}}^{\text{GPT}}(s) = s \times A^{\text{GPT}}, \quad A^{\text{GPT}} \approx 134 \text{ Ko}. \quad (3)$$

Démonstration. On applique l'équation (2) à la couche MoE, qui est la couche dominante. En notant $d = 2880$ la dimension cachée, $H_Q = 64$, $H_{KV} = 8$, $d_h = 64$ et $k = 4$ experts actifs, on décompose les contributions :

$$A_{\text{res}} = d \times p = 2880 \times 2 = 5\,760 \text{ o}$$

$$A_{\text{attn}} = (H_Q d_h + 2 H_{KV} d_h + d) \times p = (4096 + 1024 + 2880) \times 2 \approx 16\,000 \text{ o}$$

$$A_{\text{norm}} = 2 \times 2 \times d \times p = 4 \times 2880 \times 2 = 23\,040 \text{ o}$$

$$A_{\text{MoE}} = k \times (2 d_{ff} + d_{ff} + d) \times p = 4 \times (2 \times 2880 + 2880 + 2880) \times 2 = 92\,160 \text{ o}$$

Le facteur $2 d_{ff}$ dans A_{MoE} correspond aux projections gate et up de SwiGLU, d_{ff} à l'activation intermédiaire et d à la projection down. Le total donne

$$A^{\text{GPT}} = A_{\text{res}} + A_{\text{attn}} + A_{\text{norm}} + A_{\text{MoE}} = 5\,760 + 16\,000 + 23\,040 + 92\,160 = 136\,960 \text{ o} \approx 134 \text{ Ko}. \quad \square$$

5.1.2 Qwen3.6-35B-A3B

Proposition 5.4 (Pic d'activation pour Qwen3.6-35B-A3B). *Le modèle possède deux types de couches de nature différente. Soit $d = 2048$, $H_Q = 16$, $H_{KV} = 2$, $d_h = 256$ pour les couches full*

attention, et $N_k = 16$, $N_v = 32$, $d_h^{\text{lin}} = 128$ pour les couches linéaires. Avec $k = 8$ experts actifs et $d_{ff} = 512$ par expert, le pic d'activation est atteint sur la couche linéaire :

$$M_{\text{act}}^{\text{Qwen}}(s) = s \times A^{\text{Qwen}}, \quad A^{\text{Qwen}} \approx 103 \text{ Ko.} \quad (4)$$

Démonstration. On calcule séparément les deux types de couches.

Couche full attention.

$$\begin{aligned} A_{\text{res}} &= d \times p = 4\,096 \text{ o} \\ A_{\text{attn}}^{\text{full}} &= (H_Q d_h + 2 H_{KV} d_h + d) \times p = (4096 + 1024 + 2048) \times 2 = 14\,336 \text{ o} \\ A_{\text{norm}} &= 4 \times d \times p = 16\,384 \text{ o} \\ A_{\text{MoE}} &= (k + 1)(2 d_{ff} + d_{ff} + d) \times p = 9 \times (1024 + 512 + 2048) \times 2 = 64\,512 \text{ o} \end{aligned}$$

Le facteur $(k + 1)$ prend en compte les $k = 8$ experts actifs plus l'expert partagé (*shared expert*). On obtient

$$A^{\text{full}} = 4\,096 + 14\,336 + 16\,384 + 64\,512 = 99\,328 \text{ o} \approx 97 \text{ Ko.}$$

Couche linéaire (Gated DeltaNet).

$$\begin{aligned} A_{\text{attn}}^{\text{lin}} &= (N_k d_h^{\text{lin}} + N_v d_h^{\text{lin}} + d) \times p \\ &= (16 \times 128 + 32 \times 128 + 2048) \times 2 = 20\,480 \text{ o} \end{aligned}$$

Les termes résidu, norm et MoE sont identiques à la couche full, d'où

$$A^{\text{lin}} = 4\,096 + 20\,480 + 16\,384 + 64\,512 = 105\,472 \text{ o} \approx 103 \text{ Ko.}$$

Puisque $A^{\text{lin}} > A^{\text{full}}$, la couche linéaire fixe le plafond :

$$A^{\text{Qwen}} = A^{\text{lin}} \approx 103 \text{ Ko.} \quad \square$$

Corollaire 5.5 (Comparaison des pics d'activation). À nombre de tokens s identique,

$$\frac{M_{\text{act}}^{\text{GPT}}(s)}{M_{\text{act}}^{\text{Qwen}}(s)} = \frac{134}{103} \approx 1.30.$$

gpt-oss-20b consomme 30% de mémoire d'activation de plus que *Qwen3.6-35B-A3B*, principalement en raison de sa dimension cachée plus grande ($d = 2880$ contre $d = 2048$).

5.2 Approximation pratique

On peut approximer les calculs par ce tableau :

Longueur du prompt	GPT-OSS-20B	Qwen3.6-35B-A3B	Interprétation du prompt
512 tokens	≈ 67 Mo	≈ 53 Mo	Un email détaillé, une courte description de tâche
1024 tokens	≈ 134 Mo	≈ 105 Mo	Une page A4, un article de blog court
2048 tokens (prefill vLLM)	≈ 268 Mo	≈ 211 Mo	2-3 pages A4, un fichier de code de 100 lignes
4096 tokens	≈ 536 Mo	≈ 422 Mo	5-6 pages A4, fichier de code de 200 lignes

TABLE 3 – Place de la mémoire d’activation en fonction du nombre de tokens du prompt

Avec vLLM, le prefill est découpé en chunks de `max_num_batched_tokens` tokens (2048 par défaut). La mémoire d’activation de prefill est donc plafonnée à cette taille de chunk, soit $\approx \mathbf{268\ Mo}$ pour GPT-OSS-20B et $\approx \mathbf{211\ Mo}$ pour Qwen3.6-35B-A3B dans la configuration par défaut — quelle que soit la longueur réelle du prompt.

De plus, grâce au corollaire 5.5 on peut déduire que GPT-OSS-20B consomme 30% de mémoire d’activation de plus que Qwen3.6-35B-A3B.

6 Mémoire du KV cache

Le coût élémentaire du KV cache par token est une notion importante de cette étude. La taille du KV cache sur le GPU dépend surtout du nombre de requêtes actives et du nombre de tokens générés simultanément. Connaître le coût mémoire d’un token permet ainsi d’estimer la consommation globale du modèle et de mieux dimensionner la VRAM disponible.

6.1 KV cache : formulation formelle

Définition 6.1 (Coût élémentaire par token). On note

$$m_{\text{tok}} = 2 H_{\text{kv}} d_h p \quad (5)$$

avec H_{KV} le nombre de têtes matrice KV, d_h la dimension d’une tête et p la précision d’un scalaire. m_{tok} est le coût mémoire d’un token dans une couche d’attention. Le facteur 2 est pour les matrices K et V .

Théorème 6.2 (Expression du KV cache). *Pour une séquence de longueur s , la mémoire totale du KV cache est*

$$M_{\text{KV}}^{\text{GPT}}(s) = m_{\text{tok}}(L_{\text{full}} s + L_{\text{sw}} \min(s, w)) \quad (6)$$

pour *gpt-oss-20b*, et

$$M_{\text{KV}}^{\text{Qwen}}(s) = m_{\text{tok}} \cdot 10 s \quad (7)$$

pour la partie *full attention* de Qwen3.6, à laquelle s’ajoutent des états récurrents fixes pour les couches *Gated DeltaNet* (qu’on négligera par la suite).

Démonstration. Cas gpt-oss-20b. Le modèle possède $L = L_{\text{full}} + L_{\text{sw}} = 24$ couches d'attention, décomposées en $L_{\text{full}} = 12$ couches de full attention et $L_{\text{sw}} = 12$ couches de sliding window de fenêtre $w = 128$ tokens.

Pour une séquence de longueur s , on distingue deux régimes.

Régime $s \leq w$. La fenêtre glissante n'est pas encore saturée : chaque couche sliding window stocke la totalité des s tokens. Le coût total est donc

$$M_{\text{KV}}^{\text{GPT}}(s) = m_{\text{tok}} L_{\text{full}} s + m_{\text{tok}} L_{\text{sw}} s = m_{\text{tok}} (L_{\text{full}} + L_{\text{sw}}) s,$$

ce qui correspond bien à $m_{\text{tok}} (L_{\text{full}} s + L_{\text{sw}} \min(s, w))$ puisque $\min(s, w) = s$.

Régime $s > w$. Les couches full attention stockent l'intégralité des s tokens, d'où un coût $m_{\text{tok}} L_{\text{full}} s$. Les couches sliding window ne conservent que les w derniers tokens ; leur coût est constant, égal à $m_{\text{tok}} L_{\text{sw}} w$. Le coût total devient

$$M_{\text{KV}}^{\text{GPT}}(s) = m_{\text{tok}} L_{\text{full}} s + m_{\text{tok}} L_{\text{sw}} w = m_{\text{tok}} (L_{\text{full}} s + L_{\text{sw}} \min(s, w))$$

puisque $\min(s, w) = w$.

La continuité en $s = w$ est immédiate :

$$\lim_{s \rightarrow w^-} M_{\text{KV}}^{\text{GPT}}(s) = m_{\text{tok}} (L_{\text{full}} + L_{\text{sw}}) w = \lim_{s \rightarrow w^+} M_{\text{KV}}^{\text{GPT}}(s).$$

Cas Qwen3.6-35B-A3B. Le modèle possède $L = 40$ couches, dont $L_{\text{full}} = 10$ couches de full attention et $L_{\text{lin}} = 30$ couches Gated DeltaNet.

Les couches Gated DeltaNet ne maintiennent pas de KV cache croissant : elles reposent sur un état récurrent de taille fixe

$$C_{\text{rec}} = L_{\text{lin}} \left(N_{\text{kv}}^{\text{lin}} d_h^{\text{lin},k} + N_{\text{v}}^{\text{lin}} d_h^{\text{lin},v} \right) \times p,$$

indépendant de s [2, 4]. Seules les $L_{\text{full}} = 10$ couches full attention contribuent à la partie variable, chacune stockant $m_{\text{tok}}^{\text{Qwen}} s$ octets avec $m_{\text{tok}}^{\text{Qwen}} = 2 H_{\text{kv}}^{\text{full}} d_h^{\text{full}} p = 2 \times 2 \times 256 \times 2 = 2048$ octets, d'où

$$M_{\text{KV}}^{\text{Qwen}}(s) = m_{\text{tok}}^{\text{Qwen}} \cdot L_{\text{full}} \cdot s = m_{\text{tok}}^{\text{Qwen}} \cdot 10 \cdot s + C_{\text{rec}},$$

à la constante C_{rec} près (≈ 360 Ko), qui est indépendante de s et absorbée dans M_{fixed} . □

Corollaire 6.3 (Approximation pour grands contextes). *Pour $s \gg w$,*

$$M_{\text{KV}}^{\text{GPT}}(s) \approx m_{\text{tok}} L_{\text{full}} s + m_{\text{tok}} L_{\text{sw}} w,$$

avec un terme constant d'environ 3 Mo pour *gpt-oss-20b*.

6.2 Approximation pratique

On utilise ici les coûts asymptotiques par token (valables pour $s \gg w$, c'est-à-dire en dehors de la fenêtre glissante) :

- GPT-OSS-20B : $2 \text{ Ko} \times 12 \text{ couches full} = 24 \text{ Ko/token}$.

— Qwen3.6-35B-A3B : $2 \text{ Ko} \times 10 \text{ couches full} = 20 \text{ Ko/token}$.

On peut ainsi comparer les deux modèles en fonction d’une fenêtre de contexte de longueur s .

Contexte s	GPT-OSS-20B	Qwen3.6-35B-A3B	Lecture
4 096 tokens	$\approx 99 \text{ Mo}$	$\approx 80 \text{ Mo}$	Qwen3.6 utilise moins de couches en attention classique.
16 384 tokens	$\approx 387 \text{ Mo}$	$\approx 320 \text{ Mo}$	L’écart persiste à longueur intermédiaire.
32 768 tokens	$\approx 771 \text{ Mo}$	$\approx 640 \text{ Mo}$	La pente mémoire reste plus favorable à Qwen3.6.
131 072 tokens	$\approx 3.07 \text{ Go}$	$\approx 2.56 \text{ Go}$	Qwen3.6 est plus avantageux sur les très longs contextes.

TABLE 4 – Estimation de la mémoire du KV cache par requête pour les deux modèles

7 Capacité multi-utilisateurs

Seule la mémoire dynamique (le KV cache) est affectée par le nombre d’utilisateurs simultanés ; les poids du modèle et l’overhead du moteur restent constants.

7.1 Modèle d’allocation statique

Dans le cas le plus simple, on suppose que chaque utilisateur reçoit une fraction équitable du budget KV disponible $M_{\text{KV}}^{\text{max}}$ et que toutes les sessions sont actives simultanément. Pour que le quota de tokens par utilisateur soit intuitif, on fait l’hypothèse d’un partage uniforme : toutes les sessions ont la même longueur active s^* .

7.1.1 Détails formels pour la capacité multi-utilisateurs

Théorème 7.1 (Condition d’épuisement mémoire). *Soit $M_{\text{tot}} = 48 \text{ GiB}$ la VRAM totale disponible. Le GPU entre en situation d’épuisement mémoire si, pour U requêtes simultanées de longueurs actives $s_1, \dots, s_U$,*

$$M_{\text{fixed}} + \sum_{i=1}^U M_{\text{KV}}(s_i) > M_{\text{tot}}, \quad (8)$$

ou de manière équivalente,

$$\sum_{i=1}^U M_{\text{KV}}(s_i) > M_{\text{KV}}^{\text{max}}, \quad (9)$$

avec $M_{\text{KV}}^{\text{max}} = M_{\text{tot}} - M_{\text{fixed}}$.

Démonstration. La mémoire totale consommée à un instant t se décompose en

$$M_{\text{used}}(t) = \underbrace{M_{\text{static}} + M_{\text{runtime}} + M_{\text{act}}}_{M_{\text{fixed}}} + \sum_{i=1}^U M_{\text{KV}}(s_i(t)),$$

où $s_i(t)$ est la longueur active de la requête i à l’instant t .

M_{fixed} est indépendant de U et de t : les poids sont chargés une fois pour toutes, le runtime est alloué au démarrage, et le pic d'activation est borné par le chunk size de vLLM.

La somme $\sum_i M_{\text{KV}}(s_i)$ est strictement croissante en t tant que des tokens sont générés. L'épuisement mémoire survient dès que $M_{\text{used}}(t) > M_{\text{tot}}$, soit

$$\sum_{i=1}^U M_{\text{KV}}(s_i) > M_{\text{tot}} - M_{\text{fixed}} = M_{\text{KV}}^{\text{max}}. \quad \square$$

Proposition 7.2 (Contexte équitable maximal par requête). *Sous l'hypothèse d'un partage uniforme $s_1 = \dots = s_U = s^*$ et dans le régime $s^* \gg w$, le contexte maximal par requête avant saturation est*

$$s_{\text{GPT}}^*(U) \approx \frac{M_{\text{KV}}^{\text{max}}}{U m_{\text{tok}} L_{\text{full}}^{\text{GPT}}}, \quad (10)$$

$$s_{\text{Qwen}}^*(U) \approx \frac{M_{\text{KV}}^{\text{max}}}{U m_{\text{tok}} L_{\text{full}}^{\text{Qwen}}}. \quad (11)$$

Démonstration. On part de la condition limite du Théorème 7.1 à égalité :

$$\sum_{i=1}^U M_{\text{KV}}(s^*) = M_{\text{KV}}^{\text{max}}.$$

Par uniformité des s_i , cela donne $U \cdot M_{\text{KV}}(s^*) = M_{\text{KV}}^{\text{max}}$.

Dans le régime $s^* \gg w$, on applique le Corollaire 6.3 :

$$M_{\text{KV}}^{\text{GPT}}(s^*) \approx m_{\text{tok}} L_{\text{full}}^{\text{GPT}} s^* + \underbrace{m_{\text{tok}} L_{\text{sw}} w}_{\approx 3 \text{ Mo}},$$

où le terme constant est négligeable devant $M_{\text{KV}}^{\text{max}} \approx 25 \text{ Go}$. On obtient alors

$$U m_{\text{tok}} L_{\text{full}}^{\text{GPT}} s^* \approx M_{\text{KV}}^{\text{max}},$$

d'où l'équation (10). Le raisonnement est identique pour Qwen3.6-35B-A3B, dont le KV cache variable se réduit aux $L_{\text{full}}^{\text{Qwen}} = 10$ couches full attention (Théorème 6.2), d'où l'équation (11). $\square$

Corollaire 7.3 (Rapport de capacité entre les deux modèles). *À budget $M_{\text{KV}}^{\text{max}}$ identique et nombre d'utilisateurs U fixé,*

$$\frac{s_{\text{Qwen}}^*(U)}{s_{\text{GPT}}^*(U)} = \frac{L_{\text{full}}^{\text{GPT}}}{L_{\text{full}}^{\text{Qwen}}} = \frac{12}{10} = 1.2.$$

Qwen3.6-35B-A3B offre un contexte par utilisateur 20% plus grand que gpt-oss-20b à budget KV équivalent, grâce à ses couches Gated DeltaNet à état récurrent fixe.

Démonstration. Il suffit de diviser membre à membre les équations (10) et (11). Le terme m_{tok} est identique pour les deux modèles (2048 octets par token par couche) et se simplifie. $\square$

7.1.2 Approximation en pratique

Dans le tableau ci-dessous, Qwen3.6-35B-A3B est supposé quantifié à 20 Go (poids fixes réduits, budget KV plus large).

Utilisateurs	GPT-OSS-20B	Qwen3.6-35B-A3B
1	max	max
10	$\approx 106\,000$ tokens	$\approx 170\,000$ tokens
20	$\approx 53\,000$ tokens	$\approx 85\,000$ tokens
50	$\approx 21\,000$ tokens	$\approx 34\,000$ tokens
100	$\approx 10\,600$ tokens	$\approx 17\,000$ tokens

TABLE 5 – Contexte maximal par utilisateur avant saturation du budget KV (allocation statique équitable). *max* désigne le contexte maximal supporté par le modèle.

Plus précisément, grâce au corollaire 7.3, Qwen3.6-35B-A3B offre un contexte par utilisateur 20% plus grand que GPT-OSS-20B à budget KV équivalent.

7.2 Impact réel du moteur d’inférence

L’allocation statique n’est pas un scénario optimal : en pratique, les sessions ne sont pas toutes actives au même moment, et le moteur d’inférence peut mutualiser les ressources.

Ollama — parallélisme statique. Ollama supporte les requêtes concurrentes, mais son modèle d’allocation reste *statique* : pour chaque slot parallèle, la totalité du contexte configuré est réservée en VRAM dès le chargement du modèle. La mémoire nécessaire est donc $\text{NUM_PARALLEL} \times \text{CONTEXT_LENGTH} \times m_{\text{tok}} \times L$, indépendamment du nombre de tokens effectivement générés.

vLLM — allocation dynamique. vLLM utilise le *continuous batching* : plusieurs requêtes s’exécutent en parallèle et le planificateur incorpore ou retire des requêtes à chaque pas d’inférence. Grâce à PagedAttention (voir § 8), aucune mémoire n’est réservée par anticipation : les blocs KV sont alloués token par token et libérés dès la fin de la requête. Le budget KV disponible est donc utilisé bien plus efficacement que sous Ollama, ce qui permet de servir davantage d’utilisateurs simultanés avec le même matériel.

8 Ollama et vLLM

La section précédente a introduit la distinction séquentiel/batching continu. Ce tableau récapitule l’ensemble des différences fonctionnelles entre les deux moteurs, en couvrant également les fonctionnalités avancées de vLLM absentes d’Ollama.

Critère	Ollama	vLLM
Parallélisme des requêtes	Configurable, allocation KV <i>statique</i> par slot	Batching continu ; allocation KV <i>dynamique</i> à la demande
Gestion du KV cache	Allocation par session	PagedAttention : blocs virtuels, alloués à la demande [6]
Fragmentation mémoire	Élevée (allocation séquentielle)	Quasi nulle [6]
Préfixe partagé (system prompt)	Non	Prefix caching : les blocs KV d'un préfixe commun sont réutilisés entre requêtes
Décodage spéculatif	Non	Speculative decoding avec modèle brouillon (réduit la latence)
Multi-GPU	Limité (expérimental)	Tensor parallelism natif sur N GPUs
Formats de quantification	GGUF (format principal)	AWQ, GPTQ, FP8, INT4, INT8, BitsAndBytes ²
Latence mono-utilisateur	Faible	Faible
Latence multi-utilisateurs	Dégradée par défaut, améliorable mais allocation KV statique par slot	Stable grâce au batching continu [6]
Cas d'usage	Développement, test local	Production, charge multi-requêtes

TABLE 6 – Comparaison fonctionnelle entre Ollama et vLLM

2. vLLM supporte également GGUF (support expérimental) et MXFP4 sur architectures Blackwell (RTX 5090). Pour les A6000, AWQ et FP8 sont les formats recommandés.

8.1 PagedAttention

PagedAttention est la fonctionnalité clé qui distingue vLLM des autres moteurs d’inférence du point de vue de la mémoire. Classiquement, le KV cache d’une requête est alloué d’un seul coup en mémoire, à hauteur de la longueur maximale possible, ce qui est loin d’être optimal. En effet, la mémoire non encore utilisée est réservée préventivement (*réserve anticipée*) et les emplacements libérés entre deux requêtes créent de la fragmentation.

PagedAttention découpe le KV cache en blocs logiques de taille fixe (typiquement $B_s = 16$ tokens), mappés sur des blocs physiques non voisins en VRAM via une table de correspondance par requête. Un bloc n’est alloué que lorsqu’il est réellement rempli, et il est libéré dès que la requête se termine, sans aucun déplacement de mémoire.

8.1.1 PagedAttention : formulation formelle

Définition 8.1 (PagedAttention). PagedAttention découpe le KV cache en blocs de taille fixe B_s et associe à chaque requête une table de correspondance logique-vers-physique. Cette stratégie limite la fragmentation mémoire et permet une allocation dynamique des blocs [6].

Proposition 8.2 (Capacité en blocs). *Le coût d’un bloc vaut*

$$C_{\text{blk}} = B_s m_{\text{tok}}.$$

Le nombre total de blocs disponibles est

$$N_{\text{blk}} = \left\lfloor \frac{M_{\text{KV}}^{\text{max}}}{C_{\text{blk}}} \right\rfloor.$$

Démonstration. Il suffit de diviser le budget mémoire du KV cache par le coût d’un bloc de taille B_s . La partie entière traduit le fait qu’on ne peut allouer qu’un nombre entier de blocs. $\square$

Remarque 8.3. Le choix de vLLM devient particulièrement pertinent lorsque plusieurs requêtes coexistent, car le batching continu et la gestion en blocs réduisent fortement le gaspillage mémoire [6].

Cette architecture permet deux optimisations supplémentaires importantes pour un usage multi-utilisateurs :

- **Prefix caching.** Si plusieurs requêtes partagent le même préfixe (par exemple un system prompt identique), les blocs KV du préfixe ne sont calculés et stockés qu’une seule fois.
- **Partage de blocs lors du beam search.** Les branches d’un beam search qui partagent un préfixe commun pointent vers les mêmes blocs physiques, sans duplication.

9 Conclusion et Perspectives

Cette étude a permis de caractériser et de formaliser le comportement de la mémoire VRAM lors du déploiement concurrent de grands modèles de langage sur GPU sous contrainte physique de mémoire (NVIDIA RTX A6000, 48 Go).

D’une part, l’analyse comparative des architectures met en évidence un compromis crucial entre simplicité d’intégration et efficacité structurelle. Le modèle GPT-OSS-20B se distingue

par sa facilité de déploiement immédiat, due à son checkpoint compact et à sa quantification native en format **MXFP4**. À l'inverse, l'architecture hybride de **Qwen3.6-35B-A3B**, combinant des couches d'attention classique et des couches récurrentes Gated DeltaNet, démontre une efficacité supérieure pour les séquences longues et les scénarios multi-utilisateurs. En limitant la croissance linéaire du KV cache aux seules couches d'attention complète, ce modèle réduit l'empreinte mémoire dynamique par token, autorisant une allocation de contexte par utilisateur supérieure de 20% à budget de KV cache équivalent.

D'autre part, le choix du moteur d'inférence joue un rôle déterminant dans la traduction de ces caractéristiques physiques en capacité opérationnelle. Alors qu'Ollama impose une allocation statique par slot qui fragmente la VRAM et sous-exploite le matériel, vLLM résout ces limitations grâce au batching continu et à PagedAttention. En éliminant la fragmentation externe et en allouant dynamiquement les blocs de mémoire à la demande, vLLM optimise le budget disponible (M_{KV}^{max}), augmentant de manière significative la densité d'utilisateurs simultanés.

À titre de perspective, ce travail de dimensionnement théorique jette les bases d'une modélisation plus large. Une extension naturelle consisterait à intégrer la charge mémoire liée aux entrées multimodales (images et vidéos), dont la taille des tenseurs d'activation est intrinsèquement variable. De plus, l'impact des formats de quantification émergents en très basse précision (FP8 et FP4) sur le compromis perplexité-débit sous forte concurrence constituera un axe d'investigation futur majeur pour affiner nos modèles d'allocation.

Références

- [1] OPENAI et al. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv : [2508.10925](https://arxiv.org/abs/2508.10925) [cs.CL]. URL : <https://arxiv.org/abs/2508.10925>.
- [2] ALIBABA CLOUD. *Qwen3.6-35B-A3B : Agentic Coding Power, Now Open to All*. <https://huggingface.co/Qwen/Qwen3.6-35B-A3B>. Accès le 2026-05-28. 2026.
- [3] OPENAI. *config.json for openai/gpt-oss-20b*. <https://huggingface.co/openai/gpt-oss-20b/blob/main/config.json>. Accessed 2026-05-28. 2025.
- [4] ALIBABA CLOUD. *Qwen3.6-35B-A3B : Agentic Coding Power, Now Open to All*. <https://qwen.ai/blog?id=qwen3.6-35b-a3b>. Accès le 2026-05-28. 2026.
- [5] Ashish VASWANI et al. « Attention is all you need ». In : *Advances in neural information processing systems* 30 (2017).
- [6] Woosuk KWON et al. « Efficient memory management for large language model serving with PagedAttention ». In : *Proceedings of the 29th Symposium on Operating Systems Principles*. 2023, p. 611-626.

A Notations

Symbole	Sens	Valeur / remarque
L	Couches d’attention totales	$L = 24$ pour gpt-oss-20b
L_{full}	Couches full attention	12 pour gpt-oss-20b ; 10 pour Qwen3.6
L_{sw}	Couches sliding window	$L_{\text{sw}} = 12$ pour gpt-oss-20b
w	Taille de la fenêtre glissante	$w = 128$ tokens
H_{kv}	Têtes KV	8 pour gpt-oss-20b ; 2 pour Qwen3.6
d_h	Dimension d’une tête	64 pour gpt-oss-20b ; 256 pour Qwen3.6
p	Taille d’un scalaire en mémoire	$p = 2$ octets en F16/BF16
s	Longueur active de séquence	$0 \leq s \leq S$
S	Longueur maximale de contexte	131 072 pour gpt-oss-20b ; 262 144 pour Qwen3.6
M_{tot}	VRAM totale	48 GB
M_{static}	Poids du modèle	Dépend du checkpoint
M_{runtime}	Overhead du moteur	Dépend du backend
M_{act}	Pic d’activation au prefill	Dépend du prompt
M_{fixed}	Coûts fixes totaux	$M_{\text{static}} + M_{\text{runtime}} + M_{\text{act}}$
M_{KV}	Mémoire du KV cache	Variable selon s et U
$M_{\text{KV}}^{\text{max}}$	Budget disponible pour le KV cache	Dépend du moteur
U	Requêtes simultanées	Variable
B_s	Taille d’un bloc PagedAttention	Typiquement 16 tokens
N_{blk}	Nombre de blocs KV disponibles	Dépend du budget

TABLE 7 – Notations utilisées dans le rapport